| EX.NO: 1(a) | STUDY OF NETWORK COMMANDS |
| --- | --- |
| Date : | |

**AIM**

To study and understand various **network commands** used for network configuration, connectivity testing, troubleshooting, and monitoring.

**Objective**

- To learn commonly used network commands.
- To identify the IP address and network configuration of a system.
- To test connectivity between computers.
- To trace the path to a remote host.
- To examine active network connections.
- To troubleshoot basic network problems.

**Prerequisite Knowledge**

- Computer networks
- IP addresses
- TCP/IP protocol
- LAN and WAN
- DNS
- Routers and switches
- Command Prompt/Terminal

**Theory**

Network commands are command-line utilities used to obtain information about a computer's network configuration and to diagnose network-related problems.

**COMMANDS**

```
arp -a
```

**Description:**
Displays the **Address Resolution Protocol (ARP) table**, showing the mapping between IP addresses and MAC addresses of devices known to the computer.

**Use:**

- Verifies IP-to-MAC address mappings.
- Helps troubleshoot local network communication.
- Can help identify unexpected devices on a local network.

```
hostname
```

**Description:**
Displays the name of the current computer.

**Use:**
Useful for identifying a particular computer in a network containing multiple hosts.

**`ipconfig`**

**Description:**
Displays the basic TCP/IP configuration of the system, including the IP address, subnet mask, and default gateway.

**Use:**
Commonly used to check the current network configuration.

**`ipconfig /all`**

**Description:**
Displays detailed TCP/IP configuration information, including the MAC address, DHCP status, DNS servers, adapter information, IP address, and gateway.

**Use:**
Used for detailed network analysis and troubleshooting IP configuration.

**`ipconfig /renew`**

**Description:**
Requests a new IP configuration from the DHCP server.

**Use:**
Useful when troubleshooting DHCP-related connectivity problems or obtaining updated network configuration.

```
ipconfig /release
```

**Description:**
Releases the current DHCP-assigned IP configuration.

**Use:**
Normally used before obtaining a new DHCP configuration or when disconnecting from a network.

**`ipconfig /flushdns`**

**Description:**
Clears the local DNS resolver cache.

**Use:**
Useful when troubleshooting problems caused by outdated or incorrect cached DNS information.

**`nbtstat -a`**

**Description:**
Displays the NetBIOS name table of a specified computer over TCP/IP.

**Use:**
Useful for troubleshooting NetBIOS name-resolution and connectivity problems in Windows networks.

**`netdiag`**

**Description:**
`netdiag` was a Windows network diagnostic utility used to perform various network diagnostic tests.

**Use:**
It was used for diagnosing network adapters, configurations, and connectivity.

**`netstat`**

**Description:**
Displays active network connections, listening ports, and network statistics.

For numerical addresses:

```
netstat -n
```

**Use:**

- Monitors active connections.
- Identifies listening ports.
- Helps troubleshoot TCP/UDP communication.

**`nslookup`**

**Description:**
Queries DNS servers to obtain information about domain names and IP addresses.

**Command:**

```
nslookup google.com
```

**Use:**
Essential for diagnosing DNS problems and verifying domain-name resolution.

**`pathping`**

**Description:**
Combines features of `ping` and `tracert`. It identifies the route to a destination and collects packet-loss and latency information.

**Command:**

```
pathping google.com
```

**Use:**
Helps identify network links or routers where packet loss or delays are occurring.

**`ping`**

**Description:**
Sends ICMP Echo Request messages to a destination and waits for Echo Replies.

**Command:**

```
ping google.com
```

**Use:**

- Tests network connectivity.
- Determines whether a host is reachable.
- Measures approximate round-trip response time.

**route**

**Description:**
Displays and, when permitted, modifies the local IP routing table.

**Command:**

```
route print
```

**Use:**

- Examines routing information.
- Helps understand packet forwarding.
- Can be used to configure static routes.

**tracert**

**Description:**
Traces the path taken by packets to reach a destination by displaying intermediate network hops.

**Command:**

```
tracert google.com
```

**Use:**

- Identifies intermediate routers.
- Helps locate routing problems.
- Helps identify network hops with high delay.

**TCPDUMP COMMANDS**

`tcpdump` is a command-line packet analyzer commonly available on **Linux/Unix-like systems**.
It captures and analyzes network traffic.

## a) Capture packets from all interfaces

```
tcpdump -i any
```

**Use:**
Captures packets from all available interfaces supported by the system.

## b) Capture packets from a specific interface

```
tcpdump -i eth0
```

**Use:**
Captures traffic only from the specified network interface.

## c) Read packets from a saved capture file

```
tcpdump -r packets_file
```

**Use:**
Allows previously captured packets to be analyzed offline.

## d) Capture traffic from a network

```
tcpdump net 192.168.1.0/24
```

**Use:**
Captures traffic associated with the specified IP network.

## e) Capture packets from a specific source

```
tcpdump src 192.168.1.100
```

**Use:**
Captures packets whose source matches the specified IP address.

## f) Capture packets destined for a specific host

```
tcpdump dst 192.168.1.100
```

**Use:**
Captures packets whose destination matches the specified IP address.

## g) Capture SSH traffic

```
tcpdump ssh
```

**Use:**
Filters traffic associated with SSH when the installed tcpdump supports the `ssh` protocol name.

## h) Capture traffic on port 22

```
tcpdump port 22
```

**Use:**
Captures traffic using port 22, commonly associated with SSH.

## i) Capture traffic within a port range

```
tcpdump portrange 22-125
```

**Use:**
Captures traffic using ports from 22 through 125.

| S.No. | Command | Main Purpose |
|---|---|---|
| 1 | arp -a | Display ARP table |

| | | |
|---|---|---|
| 2 | hostname | Display computer name |
| 3 | ipconfig | Display basic IP configuration |
| 4 | ipconfig /all | Display detailed IP configuration |
| 5 | ipconfig /renew | Renew DHCP configuration |
| 6 | ipconfig /release | Release DHCP configuration |
| 7 | ipconfig /flushdns | Clear DNS cache |
| 8 | nbtstat -a | Display NetBIOS information |
| 9 | netdiag | Network diagnostics on older Windows systems |
| 10 | netstat | Display network connections |
| 11 | nslookup | Perform DNS queries |
| 12 | pathping | Analyze route, delay, and packet loss |
| 13 | ping | Test connectivity |
| 14 | route print | Display routing table |
| 15 | tracert | Trace network path |
| 16 | tcpdump | Capture and analyze packets |

**Advantages**

1. Easy to use from the command line.
2. Useful for network troubleshooting.
3. Provides network configuration information.
4. Helps identify connectivity problems.
5. Helps administrators monitor network connections.
6. Does not require a graphical interface.

**Disadvantages**

1. Some commands require administrator privileges.
2. Output can be difficult for beginners to understand.
3. Firewall restrictions may affect results.
4. Commands and options can differ between operating systems.

**Real-Life Applications**

- Troubleshoot Internet connectivity.
- Find IP and MAC addresses.
- Diagnose DNS problems.
- Check routing.
- Identify network delays.
- Monitor active connections.
- Locate network failures.

**Viva Questions and Answers**

**1. What is a network command?**
A network command is a command-line utility used to configure, monitor, or troubleshoot a network.

**2. Which command displays the IP address?**
ipconfig

**3. Which command tests connectivity?**
ping

**4. Which command traces the route to a destination in Windows?**
tracert

**5. Which command is used for DNS lookup?**
nslookup

**6. Which command displays the MAC address?**
getmac

**7. What does ARP do?**
ARP maps an IP address to a MAC address on a local network.

**8. Which command displays active network connections?**
netstat

**9. Which command displays the routing table?**
route print

**10. What is the purpose of pathping?**
It analyzes the path to a destination and provides information about packet loss and latency.

**Result**

Thus, various network commands were studied and their functions, syntax, and applications in network configuration, connectivity testing, troubleshooting, routing, DNS analysis, and packet monitoring were understood.

| EX.NO: 1(b) | SIMULATION OF PING COMMAND |
| --- | --- |
| Date: | |

**AIM**

To write and execute a Java program to simulate the **PING command** and check whether a destination host is reachable.

**Objective**

- To understand the working of the PING command.
- To check network connectivity between two hosts.
- To determine the response time of a destination host.

**Theory**

**PING (Packet Internet Groper)** is a network utility used to test whether a host is reachable over a network.

It sends an **ICMP Echo Request** to the destination. If the destination is reachable, an **ICMP Echo Reply** is received.

In Java, the InetAddress class provides the isReachable() method, which can be used for a basic simulation of the ping operation.

**Advantages**

1. Simple and easy to implement.
2. Helps understand network connectivity.
3. Measures approximate response time.
4. Useful for basic network troubleshooting.

**Disadvantages**

1. isReachable() is not a complete replacement for the operating-system ping utility.
2. Results depend on the operating system and network configuration.
3. Firewalls may prevent responses.
4. It does not provide all ICMP packet details.

**Applications**

- Testing network connectivity.
- Checking whether a server is reachable.
- Troubleshooting network problems.
- Measuring approximate network response time.
- Testing local and remote hosts.

**Algorithm**

1. Start the program.
2. Read the destination host name or IP address.
3. Obtain the IP address using InetAddress.
4. Send a reachability request.
5. Record the start and end time.
6. Calculate the response time.
7. Display the result.
8. Repeat the process four times.
9. Stop the program.

**JAVA PROGRAM (PING SIMULATION)**

```
import java.net.InetAddress;
import java.util.Scanner;
public class PingSimulation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter host name or IP address: ");
        String host = sc.nextLine();
        try {
            InetAddress address = InetAddress.getByName(host);
            System.out.println("\nPinging " + host + " ["
                    + address.getHostAddress() + "]");
            for (int i = 1; i <= 4; i++) {
                long start = System.currentTimeMillis();
                if (address.isReachable(5000)) {
                    long end = System.currentTimeMillis();
                    System.out.println("Reply from "
                            + address.getHostAddress()
                            + ": time=" + (end - start) + " ms");
```

```
            } else {
                System.out.println("Request timed out.");
            }
            Thread.sleep(1000);
        }
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
    sc.close();
  }
}
```

**Run**

Javac PingSimulation.java
java PingSimulation

**Input**

Enter host name or IP address: google.com

**Output**

Pinging google.com [64.233.181.113]

Request timed out.

Request timed out.

Request timed out.

Request timed out.

**Viva Questions and Answers**

**1. What is PING?**

PING is a network utility used to check whether a host is reachable.

**2. What does PING stand for?**

Packet Internet Groper.

**3. Which protocol is normally used by PING?**

ICMP — Internet Control Message Protocol.

**4. What is an Echo Request?**

It is an ICMP message sent to check whether a destination is reachable.

**5. What is an Echo Reply?**

It is the response sent by the destination to an ICMP Echo Request.

**6. Which Java class is used in this program?**

InetAddress.

**7. Which method is used to check reachability?**

isReachable().

**8. What does getHostAddress() return?**

It returns the IP address of the host.

**9. What does "Request timed out" mean?**

It means that a response was not received within the specified time.

**10. What is response time?**

It is the approximate time taken to receive a response from the destination.

**Result**

Thus, the Java program to simulate the PING command was implemented and executed successfully.

| EX.NO: 1(c) | SIMULATION OF TRACEROUTE COMMAND |
| --- | --- |
| Date : | |

**AIM**

To write a Java program to simulate the **Traceroute (tracert) command** and display the intermediate routers/hops between the source computer and a destination host.

**Objective**

- To understand the working of the traceroute command.
- To identify the intermediate network hops.
- To determine the IP address and response time of each hop.
- To understand how packets travel from source to destination.

**Theory**

**Traceroute** is a network diagnostic command used to determine the path followed by packets from a source computer to a destination.

In Windows, the command is called:

tracert

In Linux/macOS, it is commonly called:

traceroute

Traceroute works by sending packets with gradually increasing **TTL (Time To Live)** values. When the TTL becomes zero at an intermediate router, the router returns a response. By recording these responses, the path through the network can be identified.

**Advantages**

1. Easy to implement using Java.
2. Helps understand network routing.
3. Can identify whether a destination is reachable.
4. Measures approximate response time.
5. Useful for basic network troubleshooting.

**Disadvantages**

1. Java's InetAddress.isReachable() does **not provide a true hop-by-hop traceroute**.
2. It may not reveal every intermediate router.
3. Results depend on operating-system permissions and network configuration.
4. Firewalls may block ICMP or related responses.
5. Response time can vary depending on network traffic.

**Applications**

- Network troubleshooting.
- Finding routing problems.

- Measuring network response time.
- Studying computer networks.
- Identifying connectivity problems.

**Real-Life Example**

When a user cannot access a website, traceroute can help determine where the connection is failing:

Computer
↓
Router
↓
ISP Router
↓
Internet Router
↓
Destination Server

By examining the hops, a network administrator can identify possible routing or connectivity problems.

**ALGORITHM**

1. Start the program.
2. Read the destination host name or IP address from the user.
3. Find the IP address of the destination.
4. Set the maximum number of hops.
5. For each hop:
    - Attempt to determine a reachable address.
    - Measure the response time.
    - Display the hop number, IP address, and response time.
6. Continue until the destination is reached or the maximum hop count is exceeded.
7. Stop the program.

**JAVA PROGRAM (TRACEROUTE)**

```
import java.net.InetAddress;

import java.util.Scanner;

public class TracerouteSimulation {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter destination host name or IP address: ");
```

```
String host = sc.nextLine();
try {
    InetAddress destination = InetAddress.getByName(host);
    System.out.println();
    System.out.println("Tracing route to " + host +
            " [" + destination.getHostAddress() + "]");
    System.out.println("Maximum number of hops: 10");
    System.out.println();
    for (int ttl = 1; ttl <= 10; ttl++) {
        long start = System.currentTimeMillis();
        boolean reachable = destination.isReachable(3000);
        long end = System.currentTimeMillis();
        System.out.print(ttl + "    ");
        if (reachable) {
            System.out.println(
                destination.getHostAddress() +
                "    " + (end - start) + " ms"
            );
            System.out.println();
            System.out.println("Trace complete.");
            break;
        } else {
            System.out.println("*    Request timed out.");
        }
    }
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
```

```
        }
        sc.close();
    }
}
```

**Run**

javac TracerouteSimulation.java
java TracerouteSimulation

**Input**

Enter destination host name or IP address: google.com

**Output**

Enter destination host name or IP address: google.com

Tracing route to google.com [142.251.184.101]

Maximum number of hops: 10

1 * Request timed out.

2 * Request timed out.

3 * Request timed out.

4 * Request timed out.

5 * Request timed out.

6 * Request timed out.

7 * Request timed out.

8 * Request timed out.

9 * Request timed out.

10 * Request timed out.

**Viva Questions and Answers**

**1. What is traceroute?**

Traceroute is a network diagnostic tool used to determine the path taken by packets to reach a destination.

**2. What is the Windows command for traceroute?**

tracert

**3. What is TTL?**

TTL stands for **Time To Live**. It limits the lifetime of a packet in a network.

**4. Why is traceroute useful?**

It helps identify intermediate routers and locate network connectivity problems.

**5. Which Java class is used in this program?**

InetAddress from the java.net package.

**6. Which method checks whether a host is reachable?**

isReachable().

**7. What does getHostAddress() return?**

It returns the IP address of the host.

**8. What does getByName() do?**

It obtains an InetAddress object for a specified host name or IP address.

**9. What does * indicate in traceroute?**

It generally indicates that a response was not received within the specified timeout.

**10. Is this program exactly the same as the operating-system traceroute command?**

No. It is a **simulation/basic Java implementation**. A true traceroute requires controlling TTL values and processing ICMP/UDP responses for each individual hop.

**Result**

Thus, the Java program to simulate the Traceroute command was implemented and executed successfully**.**

| EX. NO : | CREATE SOCKET FOR HTTP FOR WEB PAGE UPLOAD AND DOWNLOAD |
| --- | --- |
| Date : | |

**AIM**

To write a Java HTTP web client program that downloads a web page using TCP sockets.

**Objectives**

- Understand TCP socket communication.
- Establish a connection with a web server.
- Send an HTTP request.
- Receive an HTTP response.
- Display/download the webpage.

**Prerequisite Knowledge**

- TCP/IP.
- Client-server architecture.
- Socket programming.
- HTTP protocol.
- Java I/O streams.

**Theory / Concept**

HTTP operates as an application-layer protocol. A web client establishes a TCP connection with a web server and sends an HTTP request.

Typical request:

GET / HTTP/1.1
Host: example.com
Connection: close

The server responds with an HTTP response containing:

- Status line
- Headers
- Webpage content

**Advantages**

- Reliable communication through TCP.
- Simple client-server architecture.
- Useful for understanding HTTP internally.

**Disadvantages**

- Plain HTTP does not provide encryption.

- Manual HTTP parsing is more complex than using high-level libraries.
- Modern websites may require HTTPS.

**Applications**

- Web browsers.
- Web crawlers.
- Testing tools.
- HTTP learning applications.
- Network programming.

**Real-Time Example**

A browser connects to a web server and sends a GET request to retrieve the homepage.

**ALGORITHM**

1. Create a TCP socket.
2. Connect to the web server on port 80.
3. Create an HTTP GET request.
4. Send the request.
5. Read the server response.
6. Display the response.
7. Close the socket.

**PROGRAM**

```
import java.io.*;
import java.net.*;
public class HTTPClient {
   public static void main(String[] args) {
      String host = "example.com";
      try {
         Socket socket = new Socket(host, 80);
         PrintWriter out = new PrintWriter(
               socket.getOutputStream(), true);
         BufferedReader in = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));
         out.println("GET / HTTP/1.1");
         out.println("Host: " + host);
         out.println("Connection: close");
         out.println();
```

```
            String response;
            while ((response = in.readLine()) != null) {
                System.out.println(response);
            }
            socket.close();
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

**Run**

javac HTTPClient.java

java HTTPClient

**Input**

Host: example.com

Port: 80

Request: GET /

**Output**

HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: ...

<!doctype html>

<html>

...

</html>

## Viva Questions

1. **What is HTTP?**

   HTTP stands for HyperText Transfer Protocol.

2. **Which transport protocol does HTTP normally use?**

   TCP for traditional HTTP/1.x and HTTP/2.

3. **What is a socket?**

   A socket is an endpoint for network communication.

4. **What is port 80?**

   The conventional port for HTTP.

5. **What is GET?**

   An HTTP method used to request a resource.

6. **What is a web server?**

   A system that serves web resources to clients.

7. **What is an HTTP response?**

   The response returned by a server after receiving an HTTP request.

8. **What is TCP?**

   A reliable, connection-oriented transport protocol.

9. **What is localhost?**

   The local computer, commonly represented by 127.0.0.1.

10. **What does Connection: close indicate?**

    The TCP connection should be closed after the response.

## Result

Thus, an HTTP web client was successfully implemented using Java TCP sockets.

| EX.NO: 3(a) | TCP Socket Applications – Echo and Chat |
|---|---|
| Date: | |

**AIM**

To implement an Echo Client and Echo Server using TCP sockets.

**Objectives**

- Understand TCP client-server communication.
- Establish a TCP connection.
- Exchange messages between client and server.
- Understand input/output streams.

**Theory**

An echo server sends back the same message received from a client.

Client → Message → Server
Client ← Same Message ← Server

**Advantages**

- Simple to implement.
- Demonstrates TCP communication.
- Useful for learning sockets.

**Disadvantages**

- Not suitable for large-scale communication.
- Requires a running server.
- Basic echo service provides no additional processing.

**Applications**

- Socket testing.
- Network debugging.
- Teaching client-server communication.

**Real-Time Example**

A network engineer uses an echo service to verify that data sent over a connection is received correctly.

**ALGORITHM – SERVER**

1. Create a server socket.
2. Wait for a client.
3. Accept the connection.
4. Read client data.
5. Send the same data back.

6. Close the connection.

**ALGORITHM – CLIENT**

1. Create a socket.
2. Connect to server.
3. Read user message.
4. Send message.
5. Receive echoed message.
6. Display the message.
7. Close socket.

**JAVA ECHO SERVER**

```
import java.io.*;
import java.net.*;
public class EchoServer {
   public static void main(String[] args) {
      try {
         ServerSocket serverSocket = new ServerSocket(5000);
         System.out.println("Server waiting...");
         Socket socket = serverSocket.accept();
         BufferedReader in = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));
         PrintWriter out = new PrintWriter(
               socket.getOutputStream(), true);
         String message;
         while ((message = in.readLine()) != null) {
            if (message.equalsIgnoreCase("exit"))
               break;
            out.println("Echo: " + message);
         }
         socket.close();
         serverSocket.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

**JAVA ECHO CLIENT**

```
import java.io.*;
import java.net.*;
public class EchoClient {
   public static void main(String[] args) {
      try {
         Socket socket = new Socket("localhost", 5000);
         BufferedReader keyboard = new BufferedReader(
               new InputStreamReader(System.in));
         BufferedReader in = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));
         PrintWriter out = new PrintWriter(
               socket.getOutputStream(), true);
         String message;
         while (true) {
            System.out.print("Enter message: ");
            message = keyboard.readLine();
            out.println(message);
            if (message.equalsIgnoreCase("exit"))
               break;
            System.out.println(in.readLine());
         }
         socket.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

**Run**

## Terminal 1:

```
javac EchoServer.java
java EchoServer
```

Terminal 2:

```
javac EchoClient.java
java EchoClient
```

**Input**

Hello
Computer Networks
exit

**Output**

Echo: Hello
Echo: Computer Networks

**Viva question**

**1. What is an Echo Server?**
An Echo Server is a server that sends the same message received from the client back to the client.

**2. What is an Echo Client?**
An Echo Client is a client program that sends a message to the server and receives the same message back.

**3. What is TCP?**
TCP stands for **Transmission Control Protocol**. It is a connection-oriented and reliable transport-layer protocol.

**4. What is a socket?**
A socket is an endpoint used for communication between two applications over a network.

**5. Which Java class is used to create a TCP server?**
`ServerSocket` is used to create a TCP server.

**6. Which Java class is used to create a TCP client?**
`Socket` is used to create a TCP client.

**7. What is the purpose of `ServerSocket`?**
`ServerSocket` listens for incoming client connection requests.

**8. What is the purpose of `Socket`?**
`Socket` establishes communication between the client and server.

**9. What is the use of `accept()`?**
The `accept()` method waits for a client and accepts the incoming connection.

**10. What is a port number?**
A port number identifies a particular application or service running on a computer.

**Result**

The **TCP Echo Client-Server** application was successfully implemented using Java sockets. The client sent messages to the server and received the same messages as responses.

| EX.NO:3(b) | TCP CHAT APPLICATION |
| --- | --- |
| Date : | |

**AIM**

To implement a simple TCP-based client-server chat application.

**Concept**

Client ←→ TCP Connection ←→ Server

The client and server exchange text messages using TCP sockets.

**Applications**

- Messaging systems.
- Customer support.
- Collaborative applications.
- Network programming practice.

**Algorithm – Chat Server**

1. Start the server.
2. Create a ServerSocket.
3. Wait for the client.
4. Accept the client connection.
5. Create input and output streams.
6. Read the client's message.
7. Display it.
8. Enter a reply.
9. Send the reply to the client.
10. Repeat until bye is received.
11. Close the connection.

**Algorithm – Chat Client**

1. Start the client.
2. Create a socket.
3. Connect to the server.
4. Create input and output streams.
5. Enter a message.
6. Send it to the server.
7. Receive the server's reply.
8. Display the reply.
9. Continue communication.
10. Close the connection when bye is entered.

**JAVA PROGRAM – CHAT SERVER**

```
import java.io.*;
import java.net.*;
public class ChatServer {
   public static void main(String[] args) {
      int port = 5001;
      try {
         ServerSocket serverSocket = new ServerSocket(port);
         System.out.println("Chat Server started...");
         System.out.println("Waiting for client...");
         Socket socket = serverSocket.accept();
         System.out.println("Client connected.");
         BufferedReader in = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));
         BufferedReader keyboard = new BufferedReader(
               new InputStreamReader(System.in));
         PrintWriter out = new PrintWriter(
               socket.getOutputStream(), true);
         String clientMessage;
         String serverMessage;
         while (true) {
            clientMessage = in.readLine();
            if (clientMessage == null ||
                  clientMessage.equalsIgnoreCase("bye")) {
               System.out.println("Client ended the chat.");
               break;
            }
            System.out.println("Client: " + clientMessage);
            System.out.print("Server: ");
            serverMessage = keyboard.readLine();
            out.println(serverMessage);
            if (serverMessage.equalsIgnoreCase("bye")) {
               break;
            }
         }
socket.close();
```

```
            serverSocket.close();
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

**JAVA PROGRAM – CHAT CLIENT**

```
import java.io.*;
import java.net.*;
public class ChatClient {
    public static void main(String[] args) {
        String serverAddress = "localhost";
        int port = 5001;
        try {
            Socket socket = new Socket(serverAddress, port);
            BufferedReader keyboard = new BufferedReader(
                    new InputStreamReader(System.in));
            BufferedReader in = new BufferedReader(
                    new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(
                    socket.getOutputStream(), true);
            System.out.println("Connected to Chat Server.");
            System.out.println("Type bye to exit.");
            while (true) {
                System.out.print("Client: ");
                String message = keyboard.readLine();
                out.println(message);
                if (message.equalsIgnoreCase("bye")) {
                    break;
                }
                String response = in.readLine();
                System.out.println("Server: " + response);
                if (response.equalsIgnoreCase("bye")) {
                    break;
                }
            }
```

```
            socket.close();
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

**Execution of Chat Program**

Compile:
javac ChatServer.java
javac ChatClient.java

**Run the server:**

java ChatServer

Then open another terminal and run:

java ChatClient

**Sample Output**

**Client:**

Connected to Chat Server.
Type bye to exit.

Client: Hello
Server: Hello! Welcome to the chat.

Client: How are you?
Server: I am fine.

Client: What is TCP?
Server: TCP is a reliable connection-oriented protocol.

Client: bye

**Server:**

Chat Server started...
Waiting for client...
Client connected.

Client: Hello
Server: Hello! Welcome to the chat.

Client: How are you?
Server: I am fine.

Client: What is TCP?
Server: TCP is a reliable connection-oriented protocol.

**Viva Questions**

**1. What is a socket?**

A socket is an endpoint used for communication between two network applications.

**2. What is TCP?**

TCP is a connection-oriented and reliable transport-layer protocol.

**3. Which Java class creates a TCP server?**

`ServerSocket`.

**4. Which Java class is used by a TCP client?**

`Socket`.

**5. What does `accept()` do?**

It waits for and accepts an incoming client connection.

**6. What is the purpose of a port number?**

It identifies a specific application or service on a host.

**7. What port is used in the Echo program?**

Port `5000`.

**8. What port is used in the Chat program?**

Port `5001`.

**9. What does `localhost` mean?**

It refers to the local computer, normally using the loopback address `127.0.0.1`.

**10. What is an Echo Server?**

A server that sends the received message back to the client.

**11. What is the difference between TCP and UDP?**

TCP provides reliable, connection-oriented communication, while UDP provides connectionless communication without TCP's reliability mechanisms.

**12. Why is `PrintWriter` used?**

It is used to conveniently send text data through the socket.

**13. Why is `BufferedReader` used?**

It is used to efficiently read character-based data.

**14. How does the server know when communication is finished?**

In this program, the special message `bye` is used to terminate the chat.

**15. Can the Chat Server support multiple clients?**

The basic program handles one client at a time. Multiple clients can be supported using threads or an executor service.

**Result**

Thus, TCP-based Echo Client/Server and a basic chat application were successfully implemented.

| EX.NO : 4 | SIMULATION OF DNS USING UDP SOCKETS |
| --- | --- |
| Date: | |

**AIM**

To simulate the working of DNS using UDP sockets.

**Objectives**

- Understand DNS name resolution.
- Understand UDP socket communication.
- Implement a simple DNS client and server.
- Map domain names to IP addresses.

**Prerequisite Knowledge**

- DNS.
- UDP.
- Client-server communication.
- Java DatagramSocket and DatagramPacket.

**Theory / Concept**

DNS converts human-readable domain names into IP addresses.

Example:

www.example.com
↓
93.184.216.34

DNS normally uses UDP for many standard queries because UDP has low overhead.

**Advantages**

- Fast communication.
- Low overhead.
- Simple implementation.
- Suitable for request-response queries.

**Disadvantages**

- UDP does not guarantee delivery.
- Packets may be lost.
- Basic simulation does not implement full DNS functionality.

**Applications**

- Domain name resolution.
- Internet browsing.
- Mail server lookup.

- Service discovery.

**Real-Time Example**

When a user enters a website name into a browser, DNS helps resolve the hostname to an IP address.

**ALGORITHM**

**SERVER:**

1. Create DatagramSocket.
2. Store domain-IP mappings.
3. Receive domain name.
4. Search mapping.
5. Send corresponding IP address.
6. Repeat.

**CLIENT:**

1. Create DatagramSocket.
2. Read domain name.
3. Send domain to server.
4. Receive IP address.
5. Display result.

**JAVA DNS SERVER**

```
import java.net.*;
import java.util.*;
public class DNSServer {
   public static void main(String[] args) {
      try {
         DatagramSocket socket = new DatagramSocket(6000);
         Map<String, String> dns = new HashMap<>();
         dns.put("example.com", "93.184.216.34");
         dns.put("localhost", "127.0.0.1");
         dns.put("google.com", "142.250.72.14");

         byte[] buffer = new byte[1024];

         System.out.println("DNS Server running...");

         while (true) {
            DatagramPacket request =
                  new DatagramPacket(buffer, buffer.length);

            socket.receive(request);
```

```
            String domain = new String(
                  request.getData(), 0, request.getLength());

            String ip = dns.getOrDefault(
                  domain, "Domain Not Found");

            byte[] response = ip.getBytes();

            DatagramPacket reply =
                  new DatagramPacket(
                        response,
                        response.length,
                        request.getAddress(),
                        request.getPort());

            socket.send(reply);
         }

      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

**JAVA DNS CLIENT**

```
import java.net.*;
import java.io.*;

public class DNSClient {
   public static void main(String[] args) {
      try {
         DatagramSocket socket = new DatagramSocket();

         BufferedReader br =
               new BufferedReader(
                     new InputStreamReader(System.in));

         System.out.print("Enter domain: ");
         String domain = br.readLine();

         byte[] data = domain.getBytes();

         InetAddress address =
               InetAddress.getByName("localhost");

         DatagramPacket packet =
               new DatagramPacket(
                     data, data.length,
                     address, 6000);
```

```
            socket.send(packet);

            byte[] buffer = new byte[1024];

            DatagramPacket response =
                  new DatagramPacket(buffer, buffer.length);

            socket.receive(response);

            String ip = new String(
                  response.getData(), 0,
                  response.getLength());

            System.out.println("IP Address: " + ip);

            socket.close();

         } catch (Exception e) {
            System.out.println(e);
         }
      }
}
```

**Run**

Terminal 1:

```
javac DNSServer.java
java DNSServer
```

Terminal 2:

```
javac DNSClient.java
java DNSClient
```

**Input**

Enter domain: example.com

**Output**

IP Address: 93.184.216.34

**Viva Questions**

1. **What is DNS?**

   Domain Name System.
2. **What is the purpose of DNS?**

   It maps domain names to network addresses.
3. **Which transport protocol is commonly used for ordinary DNS queries?**

   UDP.
4. **What is UDP?**

   A connectionless transport protocol.
5. **What is DatagramSocket?**

   A Java class used for UDP communication.
6. **What is a DatagramPacket?**

   It represents a UDP packet.
7. **What is an IP address?**

   A logical address identifying a network interface.
8. **What is a domain name?**

   A human-readable name for a network resource.
9. **What happens if the domain is not found?**

   The server returns "Domain Not Found" in this simulation.
10. **Is this a complete DNS implementation?**

    No. It is a simplified educational simulation.

**Result**

Thus, DNS name resolution was successfully simulated using UDP sockets.

| EX.NO: 5 | PACKET CAPTURE AND ANALYSIS USING WIRESHARK |
|---|---|
| Date: | |

**AIM**

To capture network packets using Wireshark and examine their protocol fields.

**Objectives**

- Learn packet capture.
- Identify different protocols.
- Analyze packet headers.
- Examine TCP/IP communication.

**Prerequisite Knowledge**

- OSI model.
- TCP/IP model.
- Ethernet.
- IP, TCP, UDP and ICMP.

**Theory**

Wireshark is a network protocol analyzer that captures packets traveling through a network interface.

A packet can contain several protocol layers:

Ethernet
↓
IP
↓
TCP/UDP/ICMP
↓
Application Protocol

**Advantages**

- Detailed packet analysis.
- Supports many protocols.
- Graphical interface.
- Useful for troubleshooting.

**Disadvantages**

- Large packet captures can be difficult to analyze.
- Requires networking knowledge.
- Captured data may contain sensitive information.

**Applications**

- Network troubleshooting.
- Security analysis.
- Protocol development.
- Network education.

**Real-Time Example**

A network engineer captures TCP traffic to investigate why a web application is not responding correctly.

**ALGORITHM**

1. Install and open Wireshark.
2. Select the active network interface.
3. Start packet capture.
4. Generate traffic using ping or browser.
5. Stop capture.
6. Apply a protocol filter.
7. Select packets.
8. Examine packet headers.
9. Record observations.

**Useful Filters**

ICMP

TCP

UDP

DNS

HTTP

**JAVA PROGRAM**

Java is not required to perform packet capture because Wireshark performs the capture. A simple Java program can generate network traffic:

```
import java.net.*;
public class PacketGenerator {
   public static void main(String[] args) {
      try {
         InetAddress address =
```

```
                InetAddress.getByName("example.com");
            System.out.println("Sending ping...");
            boolean reachable =
                address.isReachable(5000);
            System.out.println(
                "Reachable: " + reachable);

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

**Run**

javac PacketGenerator.java
java PacketGenerator

Then capture packets in Wireshark.

**Input**

Host: example.com

**Output**

Sending ping...
Reachable: true

**Viva Questions**

1. **What is Wireshark?**

   A network protocol analyzer.
2. **What is packet capture?**

   Recording network packets for analysis.
3. **What is a protocol?**

   A set of communication rules.
4. **What is an Ethernet frame?**

   A data-link-layer unit used on Ethernet networks.
5. **What is an IP packet?**

   A network-layer data unit.
6. **What is a TCP segment?**

   A transport-layer data unit used by TCP.
7. **What is a display filter?**

   A Wireshark expression used to show selected packets.
8. **Give an ICMP filter.**

   icmp.
9. **Give a DNS filter.**

   dns.
10. **Why is packet analysis important?**

    It helps diagnose communication and protocol problems.

**Result**

Thus, packets were successfully captured and examined using Wireshark.

| EX.NO: 6 | SIMULATION OF ARP/RARP PROTOCOLS |
| --- | --- |
| Date: | |

**AIM**

To simulate the working of ARP/RARP protocols.

**Objectives**

- Understand IP-to-MAC address mapping.
- Understand ARP request and reply.
- Understand the historical purpose of RARP.
- Implement address lookup using Java.

**Theory**

**ARP – Address Resolution Protocol**

ARP resolves:

IP Address → MAC Address

Example:

192.168.1.10 → AA:BB:CC:DD:EE:FF

A simplified ARP process:

Host A → ARP Request → Broadcast
Host B → ARP Reply → Host A

**RARP – Reverse Address Resolution Protocol**

RARP historically provided:

MAC Address → IP Address

RARP has largely been replaced by protocols such as BOOTP and DHCP.

**Advantages**

- ARP automatically resolves local IPv4 addresses to MAC addresses.
- Important for local network communication.
- Simple request-response operation.

**Disadvantages**

- ARP does not authenticate replies.
- ARP spoofing is a security concern.
- Broadcast traffic can increase with large numbers of devices.

**Applications**

- Local-area networking.
- IPv4 communication.
- Network troubleshooting.
- Address resolution.

**Real-Time Example**

When a computer wants to send an IPv4 packet to another device on the same LAN, it may need the destination MAC address. ARP provides that mapping.

**ALGORITHM**

**ARP:**

1. Read IP address.
2. Search IP-MAC table.
3. If found, return MAC address.
4. Otherwise generate an ARP request.
5. Receive ARP reply.
6. Store mapping.

**RARP simulation:**

1. Read MAC address.
2. Search MAC-IP table.
3. Return associated IP address.

**JAVA PROGRAM**

```
import java.util.*;
public class ARPRARP {
   public static void main(String[] args) {
      Map<String, String> arp = new HashMap<>();
      arp.put("192.168.1.1", "AA:BB:CC:DD:EE:01");
      arp.put("192.168.1.2", "AA:BB:CC:DD:EE:02");
      arp.put("192.168.1.3", "AA:BB:CC:DD:EE:03");
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter IP address: ");
      String ip = sc.nextLine();
      if (arp.containsKey(ip)) {
         System.out.println(
               "MAC Address: " + arp.get(ip));
```

```
        } else {
            System.out.println("ARP Entry Not Found");
        }
        System.out.println("\nRARP Simulation");
        System.out.print("Enter MAC address: ");
        String mac = sc.nextLine();
        String foundIP = null;
        for (Map.Entry<String, String> entry : arp.entrySet()) {
            if (entry.getValue().equalsIgnoreCase(mac)) {
                foundIP = entry.getKey();
                break;
            }
        }

        if (foundIP != null)
            System.out.println("IP Address: " + foundIP);
        else
            System.out.println("RARP Entry Not Found");

        sc.close();
    }
}
```

**Run**

```
javac ARPRARP.java
java ARPRARP
```

**Input**

```
Enter IP address: 192.168.1.2
Enter MAC address: AA:BB:CC:DD:EE:03
```

**Output**

```
MAC Address: AA:BB:CC:DD:EE:02

RARP Simulation
IP Address: 192.168.1.3
```

**Viva Questions**

1. **What is ARP?**

   Address Resolution Protocol.
2. **What does ARP resolve?**

   IPv4 address to MAC address.
3. **What is a MAC address?**

   A link-layer hardware address.
4. **What is an ARP request?**

   A request asking which device owns a particular IP address.
5. **What is an ARP reply?**

   A response containing the corresponding MAC address.
6. **What is RARP?**

   Reverse Address Resolution Protocol.
7. **What does RARP provide?**

   Historically, MAC-to-IP address mapping.
8. **What replaced RARP in many uses?**

   BOOTP and DHCP.
9. **Is ARP used across the Internet?**

   ARP operates on local IPv4 networks.
10. **What is ARP spoofing?**

    A technique in which forged ARP messages associate incorrect MAC/IP mappings.

**Result**

Thus, the working of ARP and RARP was successfully simulated.

| EX.NO: 7 | NETWORK SIMULATOR AND CONGESTION CONTROL ALGORITHMS |
|---|---|
| Date: | |

**AIM**

To study a network simulator and simulate congestion-control behavior.

**Objectives**

- Understand network simulation.
- Create nodes and links.
- Generate traffic.
- Observe packet transmission.
- Study congestion and packet loss.

**Prerequisite Knowledge**

- TCP/IP.
- Network topology.
- Queuing.
- Congestion.
- Packet transmission.

**Theory**

A **network simulator** models network behavior without requiring a physical network.

Examples include:

- NS-2
- NS-3

Congestion occurs when the amount of traffic entering a network is greater than its ability to process and forward the traffic.

Important TCP congestion-control concepts include:

- Slow Start
- Congestion Avoidance
- Fast Retransmit
- Fast Recovery

**Advantages**

- Low-cost experimentation.
- No physical hardware required.
- Repeatable experiments.
- Useful for protocol research.

**Disadvantages**

- Simulation may not perfectly represent real networks.
- Requires simulator knowledge.
- Configuration can be complex.

**Applications**

- Protocol research.
- Network performance evaluation.
- Congestion studies.
- Academic experiments.

**Real-Time Example**

A network engineer can simulate increasing traffic loads to determine how a network behaves before deploying a similar configuration in production.

**ALGORITHM**

1. Create network nodes.
2. Create links between nodes.
3. Configure bandwidth and delay.
4. Create traffic sources.
5. Generate packets.
6. Increase traffic.
7. Monitor packet loss and delay.
8. Analyze congestion.
9. Compare results.

**JAVA PROGRAM**

```
import java.util.*;

public class CongestionSimulation {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter incoming packets per second: ");
        int incoming = sc.nextInt();


        System.out.print("Enter router capacity per second: ");
```

```
        int capacity = sc.nextInt();

        if (incoming <= capacity) {
            System.out.println("Network Status: No Congestion");
            System.out.println("Packets Lost: 0");
        } else {
            int lost = incoming - capacity;

            System.out.println("Network Status: Congestion");
            System.out.println("Packets Lost: " + lost);
        }

        sc.close();
    }
}
```

**Run**

javac CongestionSimulation.java

java CongestionSimulation

**Input**

Incoming packets: 120

Router capacity: 100

**Output**

Network Status: Congestion

Packets Lost: 20

**Viva Questions**

1. **What is congestion?**

   A condition where network traffic exceeds available resources.
2. **What is a network simulator?**

   Software used to model network behavior.
3. **Name two network simulators**.

   NS-2 and NS-3.
4. **What is packet loss?**

   Failure of packets to reach their destination.
5. **What is congestion control?**

   Techniques used to prevent or reduce network congestion.
6. **What is TCP slow start?**

   A mechanism that gradually increases the congestion window.
7. **What is congestion window?**

   A TCP control variable limiting data in flight based on congestion.
8. **What is bandwidth?**

   The data-carrying capacity of a communication link.
9. **What is delay?**

   The time taken for data to travel through the network.
10. **Why use simulation?**

    To study network behavior without requiring a physical network.

**Result**

Thus, network simulation concepts and congestion behavior were successfully studied.

| EX.NO:8 | STUDY OF TCP/UDP PERFORMANCE USING SIMULATION |
|---|---|
| Date: | |

**AIM**

To study and compare the performance characteristics of TCP and UDP using simulation.

**Objectives**

- Understand TCP and UDP.
- Compare reliability and overhead.
- Study throughput and delay.
- Understand packet loss behavior.

**Theory**

| Feature | TCP | UDP |
|---|---|---|
| Connection | Connection-oriented | Connectionless |
| Reliability | Reliable | No delivery guarantee |
| Ordering | Ordered | No ordering guarantee |
| Flow Control | Yes | No |
| Congestion Control | Yes | No built-in mechanism |
| Overhead | Higher | Lower |

**Advantages of TCP**

- Reliable.
- Ordered delivery.
- Flow control.
- Congestion control.

**Disadvantages of TCP**

- Higher overhead.
- Connection establishment.
- Can introduce latency.

**Advantages of UDP**

- Low overhead.
- Fast.
- No connection establishment.

**Disadvantages of UDP**

- No built-in reliability.
- No ordering.
- Applications must handle reliability if required.

**Applications**

**TCP:**

- Web applications.
- File transfer.
- Email.

**UDP:**

- DNS.
- Streaming.
- Online gaming.
- Voice/video applications.

**Real-Time Example**

Video conferencing often benefits from low latency, while file transfer generally requires reliable delivery.

**ALGORITHM**

1. Define number of packets.
2. Define packet size.
3. Simulate TCP transmission.
4. Apply reliability/overhead assumptions.
5. Simulate UDP transmission.
6. Calculate transmitted data.
7. Compare results.
8. Display performance.

**JAVA PROGRAM**

```
import java.util.*;

public class TCPUDPPerformance {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of packets: ");
        int packets = sc.nextInt();

        System.out.print("Enter packet size (bytes): ");
        int size = sc.nextInt();
```

```
        int tcpOverhead = 20;
        int udpOverhead = 8;

        int tcpBytes =
                packets * (size + tcpOverhead);

        int udpBytes =
                packets * (size + udpOverhead);

        System.out.println("\nPerformance Simulation");

        System.out.println(
                "TCP total bytes: " + tcpBytes);

        System.out.println(
                "UDP total bytes: " + udpBytes);

        if (tcpBytes > udpBytes)
           System.out.println(
                   "UDP has lower simulated protocol overhead.");


        sc.close();
    }
}
```

**Run**

javac TCPUDPPerformance.java

java TCPUDPPerformance

**Input**

Enter number of packets: 100

Enter packet size (bytes): 1000

**Output**

Performance Simulation

TCP total bytes: 102000

UDP total bytes: 100800

UDP has lower simulated protocol overhead.

**Viva Questions**

1. **What is TCP?**

   A reliable connection-oriented transport protocol.

2. **What is UDP?**

   A connectionless transport protocol.

3. **Which is faster in terms of protocol overhead?**

   UDP generally has lower protocol overhead.

4. **Which protocol provides reliable delivery?**

   TCP.

5. **Does UDP guarantee delivery?**

   No.

6. **Does TCP provide congestion control?**

   Yes.

7. **What is throughput?**

   The amount of useful data delivered per unit time.

8. **What is latency?**

   The delay experienced during communication.

9. **Give an application of UDP**.

   DNS or real-time media.

10. **Give an application of TCP.**

    File transfer or web communication.

**Result**

Thus, the performance characteristics of TCP and UDP were successfully studied and compared using simulation.

| EX.NO: 9(a) | SIMULATION OF DISTANCE VECTOR ROUTING |
| --- | --- |
| Date: | |

**AIM**

To simulate the **Distance Vector Routing Algorithm** and determine the shortest path between different nodes in a network.

**Objective**

- To understand the working principle of Distance Vector routing.
- To calculate the shortest distance from each router to every other router.
- To construct the routing table for each router.
- To understand how routers exchange routing information with their neighboring routers.

**Prerequisite Knowledge**

Students should know:

- Basic computer networking concepts.
- IP routing and routers.
- Graphs and shortest-path concepts.
- Java programming basics.
- Arrays and loops.

**Theory**

**Distance Vector Routing (DVR)** is a dynamic routing algorithm in which every router maintains a **routing table** containing:

- Destination node
- Distance/cost to destination
- Next-hop router

Each router periodically shares its distance vector with its neighboring routers.

The main equation used is:

**D(x, y) = min [C(x, v) + D(v, y)]**

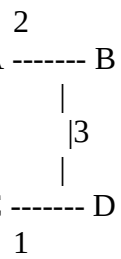
where:

- D(x,y) = distance from router x to destination y
- C(x,v) = cost from x to neighboring router v
- D(v,y) = distance from neighbor v to destination y

The algorithm repeatedly updates the routing tables until no further changes occur.

**Example Network**

Consider four routers:




Initial cost matrix:

| Router | A | B | C | D |
|---|---|---|---|---|
| **A** | 0 | 2 | 1 | ∞ |
| **B** | 2 | 0 | ∞ | 3 |
| **C** | 1 | ∞ | 0 | 1 |
| **D** | ∞ | 3 | 1 | 0 |

For example, from **A to D**:

- Direct path: ∞
- A → B → D = 2 + 3 = 5
- A → C → D = 1 + 1 = **2**

Therefore, the shortest path from A to D has cost **2** through C.

**Advantages**

1. Simple to implement.
2. Requires less computational complexity at each router.
3. Routers exchange information only with their neighbors.
4. Suitable for smaller networks.
5. Automatically adapts to changes in network topology.

**Disadvantages**

1. Routing loops can occur.
2. Suffers from the **count-to-infinity problem**.
3. Convergence can be slow.
4. Periodic routing updates consume bandwidth.
5. Not ideal for very large networks.

**Applications**

- Dynamic routing in computer networks.
- Small and medium-sized network routing.
- Routing protocols based on the distance-vector principle, such as **RIP**.

- Network topology simulation and education.

**Real-Time Example**

Suppose routers **A, B, C, and D** are connected in a network. Initially, A does not know the best route to D. After receiving routing information from its neighbor C, A discovers:

A → C = 1
C → D = 1

A → C → D = 2

Therefore, A updates its routing table and uses C as the next hop to reach D.

**Algorithm**

1. Start.
2. Read the number of routers.
3. Read the cost matrix.
4. Initialize the distance vector of every router using the cost matrix.
5. For every router i, examine every neighboring router j.
6. For every destination k, calculate:
   distance[i][j] + distance[j][k].
7. If the newly calculated distance is smaller, update the routing table.
8. Repeat the process until no distance changes occur.
9. Display the final routing tables.
10. Stop

**Java Program**

```
import java.util.Scanner;

public class DistanceVectorRouting {

   static final int INF = 999;

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter number of routers: ");
      int n = sc.nextInt();

      int[][] distance = new int[n][n];

      System.out.println("Enter the cost matrix:");
```

```
System.out.println("Enter 999 for infinity.");

for (int i = 0; i < n; i++) {
   for (int j = 0; j < n; j++) {
      distance[i][j] = sc.nextInt();
   }
}

boolean updated;

// Distance Vector Algorithm
do {
   updated = false;

   for (int i = 0; i < n; i++) {

      for (int j = 0; j < n; j++) {

         if (i == j || distance[i][j] == INF)
            continue;

         for (int k = 0; k < n; k++) {

            if (distance[j][k] == INF)
               continue;

            int newDistance =
                  distance[i][j] + distance[j][k];

            if (newDistance < distance[i][k]) {
               distance[i][k] = newDistance;
               updated = true;
            }
         }
      }
   }

} while (updated);

// Display final routing tables
System.out.println("\nFinal Distance Vector Tables:");

for (int i = 0; i < n; i++) {

   System.out.println("\nRouter " + (char)('A' + i));
```

```
            System.out.println("Destination\tCost");

            for (int j = 0; j < n; j++) {

                if (distance[i][j] == INF)
                    System.out.println(
                        (char)('A' + j) + "\t\tINF");
                else
                    System.out.println(
                        (char)('A' + j) + "\t\t"
                        + distance[i][j]);
            }
        }

        sc.close();
    }
}
```

**Input**

Enter number of routers: 3

Enter the cost matrix:

Enter 999 for infinity.

1 2  999

5 999  8

999 4   7

Final Distance Vector Tables:

Router A

| Destination | Cost |
|---|---|
| A | 1 |
| B | 2 |
| C | 10 |

Router B

| Destination | Cost |
|---|---|
| A | 5 |
| B | 7 |
| C | 8 |

Router C

| Destination | Cost |
|---|---|
| A | 9 |
| B | 4 |
| C | 7 |

**Viva Questions**

**1. What is Distance Vector Routing?**
Distance Vector Routing is a dynamic routing method where routers exchange distance information with their neighboring routers.

**2. What information does a routing table contain?**
It generally contains the destination, cost/distance, and next hop.

**3. Which algorithm is associated with Distance Vector Routing?**
The **Bellman-Ford algorithm** is the basis of Distance Vector routing.

**4. What is a distance vector?**
It is a list of the minimum known distances from a router to different destinations.

**5. What is the count-to-infinity problem?**
It is a routing problem where routers repeatedly increase the metric for an unreachable destination because of incorrect or outdated routing information.

**6. What is convergence?**
Convergence is the process by which all routers reach a stable and consistent routing state.

**7. Give an example of a Distance Vector routing protocol.**
**RIP (Routing Information Protocol).**

**8. How do routers exchange information in DVR?**
Routers exchange routing information with their neighboring routers.

**9. What is the main advantage of DVR?**
It is relatively simple and easy to implement.

**10. What is the major disadvantage of DVR?**
Slow convergence and routing loops are major disadvantages.

**11. What does infinity represent in the cost matrix?**
It represents that there is no direct connection between two routers.

**12. What happens when a shorter route is found?**
The router updates its routing table with the smaller cost.

**Result**

Thus, the **Distance Vector Routing Algorithm was successfully simulated**, and the shortest distance between the routers was calculated by repeatedly exchanging and updating distance information.

| EX.NO: 9(b) | TO SIMULATE LINK STATE ROUTING ALGORITHM |
| --- | --- |
| Date: | |

**AIM**

To simulate the **Link State Routing Algorithm** and determine the shortest path between routers using **Dijkstra's shortest path algorithm**.

**Objective**

- To understand the concept of Link State Routing.
- To learn how routers maintain information about the network topology.
- To implement Dijkstra's algorithm for finding shortest paths.
- To construct a shortest-path routing table.
- To understand how Link State routing differs from Distance Vector routing.

**Prerequisite Knowledge**

- Computer networks
- IP addressing
- Routing and routing tables
- Graphs and graph traversal
- Shortest-path algorithms
- Java programming
- Dijkstra's algorithm

**Software Requirements**

- Java JDK 8 or above
- Any Java IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt/Terminal

**Hardware Requirements**

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and mouse
- Network connectivity is optional for simulation

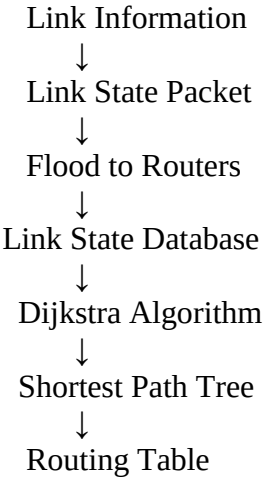
**Theory / Concept**

**Link State Routing (LSR)** is a dynamic routing technique in which every router builds a complete or sufficiently detailed map of the network topology.

1. Identifies its directly connected neighbors.
2. Determines the cost of each link.
3. Creates a **Link State Advertisement (LSA)** containing this information.
4. Floods the LSA throughout the network.
5. Builds a link-state database.

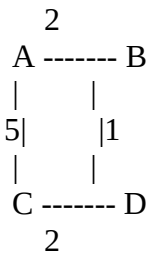
6. Runs **Dijkstra's Shortest Path First (SPF)** algorithm.
7. Creates its routing table based on the shortest paths.

**Basic Working**




**Example**

Consider the following network:




The routers are **A, B, C, and D**.

Suppose the source router is **A**.

Possible paths from A to D:

- A → B → D = 2 + 1 = **3**
- A → C → D = 5 + 2 = **7**

Therefore, the shortest path is:

A → B → D

with a total cost of **3**.

**Advantages**

1. Provides fast convergence.
2. Reduces the possibility of routing loops.
3. Each router has detailed knowledge of network topology.
4. Dijkstra's algorithm efficiently determines shortest paths.

5. Suitable for large and complex networks.
6. Routing decisions are based on complete topology information.

**Disadvantages**

1. Requires more memory to store the topology database.
2. Requires more processing power.
3. Flooding LSAs generates network overhead.
4. More complex than Distance Vector Routing.
5. Initial configuration and implementation can be complicated.

**Applications**

Link State Routing is commonly used in:

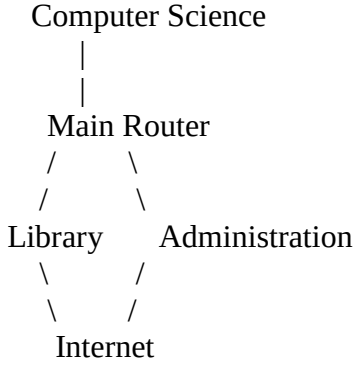
- Enterprise networks
- Large organizational networks
- Internet Service Provider networks
- Campus networks
- Data centers
- Large-scale IP networks

Examples of link-state routing protocols include:

- **OSPF – Open Shortest Path First**
- **IS-IS – Intermediate System to Intermediate System**

**Real-Time Example**

Consider a university network containing several departments:




If one link becomes unavailable, a link-state routing protocol can use its knowledge of the network topology to calculate an alternative shortest path.

For example:

Normal:
CS → Main Router → Library

If Main Router → Library fails:

CS → Main Router → Administration → Library

The routing algorithm recalculates the shortest available path.

**ALGORITHM**

**Step 1:** Start with the source router.

**Step 2:** Assign distance 0 to the source and infinity (∞) to all other routers.

**Step 3:** Mark the source router as visited.

**Step 4:** Examine all unvisited neighboring routers.

**Step 5:** Calculate the tentative distance:

New Distance = Current Distance + Link Cost

**Step 6:** If the new distance is smaller than the previously known distance, update it.

**Step 7:** Select the unvisited router with the smallest tentative distance.

**Step 8:** Mark it as visited.

**Step 9:** Repeat Steps 4–8 until all routers are visited.

**Step 10:** Display the shortest distance and shortest path from the source router.

JAVA PROGRAM

```
import java.util.*;

public class LinkStateRouting {

   static final int INF = 9999;

   static void dijkstra(int[][] graph, int source) {
      int n = graph.length;

      int[] distance = new int[n];
      int[] previous = new int[n];
      boolean[] visited = new boolean[n];
         Arrays.fill(distance, INF);
      Arrays.fill(previous, -1);

      distance[source] = 0;

      for (int count = 0; count < n - 1; count++) {

         int u = -1;
```

```
    int minDistance = INF;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && distance[i] < minDistance) {
            minDistance = distance[i];
            u = i;
        }
    }

    if (u == -1)
        break;

    visited[u] = true;

    for (int v = 0; v < n; v++) {

        if (!visited[v] &&
            graph[u][v] != 0 &&
            distance[u] != INF &&
            distance[u] + graph[u][v] < distance[v]) {

            distance[v] = distance[u] + graph[u][v];
            previous[v] = u;
        }
    }
}

System.out.println("\nRouting Table");
System.out.println("-----------------------------------");
System.out.println("Destination\tCost\tPath");
System.out.println("-----------------------------------");

for (int i = 0; i < n; i++) {

    if (i == source)
        continue;

    System.out.print((char)('A' + i) + "\t\t");

    if (distance[i] == INF) {
        System.out.println("INF\tNo Path");
    } else {
        System.out.print(distance[i] + "\t");
        printPath(previous, i, source);
        System.out.println();
    }
}
```

```
    }

    static void printPath(int[] previous, int current, int source) {

        if (current == source) {
            System.out.print((char)('A' + source));
            return;
        }

        if (previous[current] == -1) {
            System.out.print("No Path");
            return;
        }

        printPath(previous, previous[current], source);
        System.out.print(" -> " + (char)('A' + current));
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of routers: ");
        int n = sc.nextInt();

        int[][] graph = new int[n][n];

        System.out.println(
            "Enter the cost matrix (0 for no direct connection):"
        );

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = sc.nextInt();
            }
        }

        System.out.print("Enter source router (A, B, C...): ");
        char sourceChar = sc.next().toUpperCase().charAt(0);

        int source = sourceChar - 'A';

        dijkstra(graph, source);

        sc.close();
    }
}
```

**PROGRAM EXECUTION**

**Save the program as:**

LinkStateRouting.java

**Compile:**

javac LinkStateRouting.java

**Run:**

java LinkStateRouting

**Input**

Enter number of routers: 4

Enter the cost matrix (0 for no direct connection):

1 2 3 0

2 3 0 4

4 0 3 1

0 3 6 8

Enter source router (A, B, C...): a

Routing Table

---

| Destination | Cost | Path |
|---|---|---|
| B | 2 | A -> B |
| C | 3 | A -> C |
| D | 4 | A -> C -> D |

**Viva Questions and Answers**

**1. What is Link State Routing?**

**Answer:** Link State Routing is a dynamic routing technique in which routers maintain information about the network topology and calculate shortest paths.

**2. Which algorithm is used in Link State Routing?**

**Answer:** Dijkstra’s Shortest Path First algorithm is commonly used.

**3. What is a Link State Advertisement?**

**Answer:** An LSA is a message containing information about a router's directly connected links and their costs.

**4. What is flooding?**

**Answer:** Flooding is the process of distributing link-state information throughout the routing domain.

**5. What is a Link State Database?**

**Answer:** It is a database containing information about the links and topology of the network.

**6. What is the purpose of Dijkstra's algorithm?**

**Answer:** It is used to calculate the shortest path from a source router to all other reachable routers.

**7. Name a Link State routing protocol.**

**Answer:** OSPF is a commonly used Link State routing protocol.

**8. What does infinity represent in the cost matrix?**

**Answer:** Infinity represents that a destination is unreachable or there is no direct connection between two routers.

**9. What is convergence?**

**Answer:** Convergence is the process by which routers obtain consistent and updated routing information after a network change.

**10. What is the main advantage of Link State Routing?**

**Answer:** It provides fast convergence and enables routers to make routing decisions using detailed topology information.

**11. What is the major disadvantage?**

**Answer:** It requires more memory, processing power, and control-message overhead than simpler routing approaches.

**13. What is SPF?**

**Answer:** SPF stands for **Shortest Path First**.

**Result**

Thus, the Link State Routing Algorithm was successfully simulated using Dijkstra's Shortest Path First algorithm. The shortest distances and paths from the source router to all reachable routers were calculated successfully.

| EX.NO: 10 | SIMULATION OF CRC ERROR DETECTION |
| --- | --- |
| Date: | |

**AIM**

To implement and simulate the **Cyclic Redundancy Check (CRC)** error-detection technique.

**Objectives**

- Understand error detection.
- Understand polynomial division.
- Generate CRC bits.
- Detect transmission errors.
- Verify received data.

**Prerequisite Knowledge**

- Binary arithmetic.
- XOR operation.
- Data communication.
- Error detection.

**Theory / Concept**

CRC is an error-detection technique used in computer networks and data communication.

The sender:

1. Takes the original data.
2. Appends zeros.
3. Performs modulo-2 division using a generator polynomial.
4. Obtains the remainder.
5. Appends the remainder to the data.

The receiver performs the same division.

If the remainder is zero, the data is considered error-free under the CRC check.

**Advantages**

- Efficient.
- Good at detecting burst errors.
- Simple hardware implementation.
- Widely used in communication systems.

**Disadvantages**

- Detects errors but does not correct them.
- Choice of generator polynomial affects detection capability.
- Requires additional CRC bits.

**Applications**

- Ethernet.
- Data communication.
- Storage systems.
- Network frames.
- Embedded communication systems.

**Real-Time Example**

Ethernet frames include an FCS based on CRC to help detect corrupted frames.

**ALGORITHM**

1. Read data bits.
2. Read generator bits.
3. Append generator length - 1 zeros to data.
4. Perform XOR division.
5. Obtain CRC remainder.
6. Append remainder to original data.
7. At receiver, divide the transmitted codeword by generator.
8. Check remainder.
9. If remainder is zero, accept the data.
10. Otherwise report an error.

**JAVA PROGRAM**

```
import java.util.*;

public class CRC {

   static String xor(String a, String b) {

      StringBuilder result = new StringBuilder();

      for (int i = 1; i < b.length(); i++) {
         result.append(
               a.charAt(i) == b.charAt(i) ? '0' : '1');
      }

      return result.toString();
   }

   static String divide(String data, String divisor) {

      int pick = divisor.length();

      String temp = data.substring(0, pick);

      while (pick < data.length()) {

         if (temp.charAt(0) == '1')
            temp = xor(divisor, temp)
```

```
                + data.charAt(pick);
        else
            temp = xor(
                "0".repeat(divisor.length()),
                temp)
                + data.charAt(pick);

        pick++;
    }

    if (temp.charAt(0) == '1')
        temp = xor(divisor, temp);
    else
        temp = xor(
            "0".repeat(divisor.length()),
            temp);

    return temp;
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter data: ");
    String data = sc.nextLine();

    System.out.print("Enter generator: ");
    String generator = sc.nextLine();

    String appended =
        data + "0".repeat(generator.length() - 1);

    String remainder =
        divide(appended, generator);

    String transmitted =
        data + remainder;

    System.out.println(
        "CRC Remainder: " + remainder);

    System.out.println(
        "Transmitted Data: " + transmitted);

    System.out.print(
        "Enter received data: ");

    String received = sc.nextLine();

    String check =
        divide(received, generator);

    if (check.contains("1"))
```

```
            System.out.println("Error Detected");
        else
            System.out.println("No Error Detected");

        sc.close();
    }
}
```

**Run**

javac CRC.java

java CRC

**Input**

Example:

Enter data: 110101
Enter generator: 1011

The program calculates the CRC remainder and transmitted codeword.

**Output**

Example format:

CRC Remainder: 001

Transmitted Data: 110101001

Enter received data: 110101001

No Error Detected

If the received bits are changed:

Enter received data: 110101101

Error Detected

**Viva Questions**

1. **What is CRC?**

   Cyclic Redundancy Check.
2. **Is CRC an error-correction technique?**

   No. CRC is primarily an error-detection technique.
3. **What operation is used in CRC division?**

   Modulo-2 division using XOR.
4. **What is a generator polynomial?**

   A binary polynomial used to calculate the CRC remainder.
5. **What is the CRC remainder?**

   The remainder obtained from modulo-2 division.
6. **What happens when the remainder is zero?**

   The received data passes the CRC check.
7. **Can CRC correct an error?**

   No, it detects errors but does not itself correct them.
8. **Where is CRC used?**

   Ethernet and many other communication/storage systems.
9. **What is a burst error?**

   An error affecting a sequence of adjacent bits.
10. **Why is CRC better than a simple parity bit?**

    CRC can detect a much wider range of error patterns.

**Result**

Thus, the CRC algorithm was successfully implemented and used to detect transmission errors.